

Pandas

Dr. Sanjit Kumar Saha



Gisma
University
of Applied
Sciences



Pandas

- NumPy's limitations become clear when
 - We need more flexibility
 - Attaching labels to data, working with missing data, etc.
 - Attempting operations that do not map well to element-wise broadcasting
 - Groupings, pivots, etc.
- Pandas is an open-source Python library used for data manipulation, analysis and cleaning.
- It provides fast and flexible tools to work with tabular data, similar to spreadsheets or SQL tables.
- *Pandas is widely used in data science and analytics due to its integration with libraries such as:*
 - *NumPy*: numerical operations
 - *Matplotlib and Seaborn*: data visualization
 - *SciPy*: statistical analysis
 - *Scikit-learn*: machine learning workflows

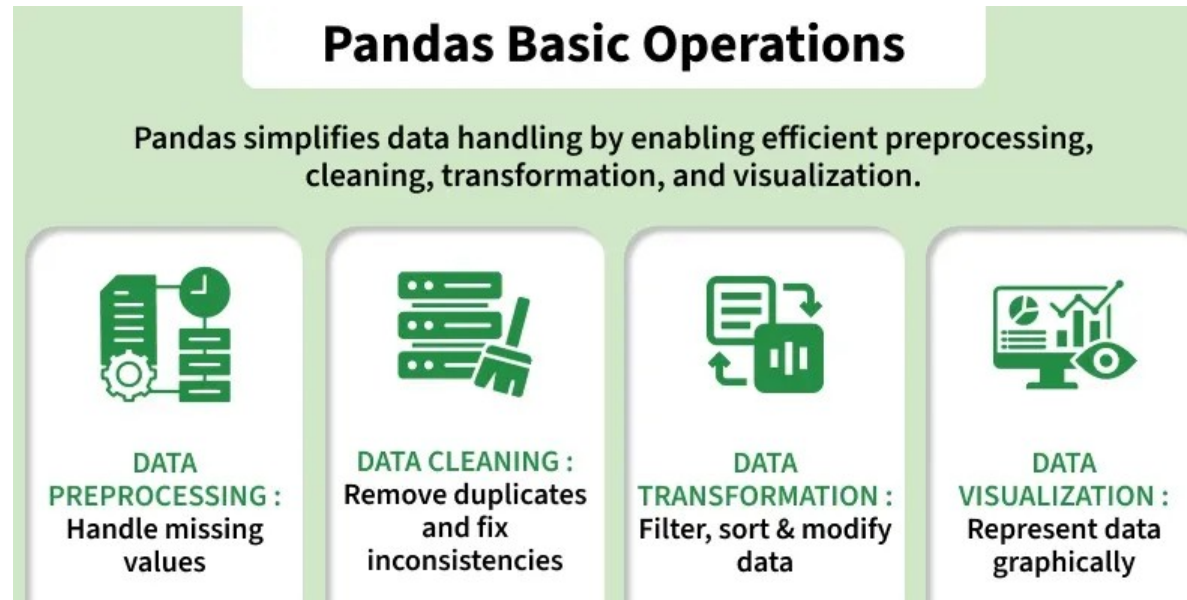


Why Use Pandas?

[VanderPlas 2016]

3

- Pandas allows efficient handling and analysis of data in a few lines of code.





Installation

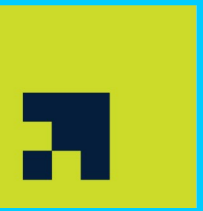
- Before using Pandas, make sure it is installed:

pip install pandas

- After the Pandas have been installed in the system we need to import the library. This module is imported using:

import pandas as pd

- *Note:* *pd* is just an alias for Pandas. It's not required but using it makes the code shorter when calling methods or properties.



Data Structures in Pandas

- Pandas provides two data structures for manipulating data which are as follows:
 1. Pandas Series
 2. Pandas DataFrame



Pandas Series

- A Pandas Series is one-dimensional labeled array capable of holding data of any type (integer, string, float, Python objects etc.).
- The axis labels are collectively called indexes.
- Series is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file.

```
import pandas as pd
import numpy as np

s = pd.Series()
print("Pandas Series: ", s)
data = np.array(['g', 'e', 'e', 'k', 's'])

s = pd.Series(data)
print("Pandas Series:\n", s)
```



Pandas Series...

```
import pandas as pd
import numpy as np

s = pd.Series()
print("Pandas Series: ", s)
data = np.array(['g', 'e', 'e', 'k', 's'])

s = pd.Series(data)
print("Pandas Series:\n", s)
```

```
Pandas Series: Series([], dtype: object)
Pandas Series:
0    g
1    e
2    e
3    k
4    s
dtype: object
```

- The values are simply a familiar *NumPy* array
 - *s.values*
- The *index* is an array-like object of type *pd.Index*
 - *s.index*
- Like with a *NumPy* array, data can be accessed by the associated index via the familiar Python square-bracket notation
 - *s[1]*
- However, the Pandas Series is much more general and flexible than the one-dimensional NumPy array that it emulates



Series as Specialized Dictionary

- A dictionary is a structure that maps arbitrary keys to a set of arbitrary values
 - `population_dict = {'California': 38332521, 'Texas': 26448193}`
- A Series is a structure that maps typed keys to a set of typed values
 - This typing is important
 - Similar to NumPy, the type information of a Pandas Series makes it much more efficient than Python dictionaries for certain operations
 - We can make the Series-as-dictionary analogy even more clear by constructing a Series object directly from a Python dictionary
 - `population = pd.Series(population_dict)`
 - Unlike a dictionary, though, the Series also supports array-style operations such as slicing

```
California    38332521
Texas         26448193
dtype: int64
```




Series as Specialized Dictionary

- Like a dictionary, the Series object provides a mapping
 - From a collection of keys to a collection of values
 - `data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])`
 - `data['b']`
- We can also use dictionary-like Python expressions and methods
 - To examine/modify the keys/indices and values
 - `'a' in data`
 - `data.keys()`
 - `list(data.items())`
 - `data['e'] = 1.25`

```
a      0.25
b      0.50
c      0.75
d      1.00
dtype: float64
```



Series as Generalized NumPy Array

- The NumPy array has an implicitly defined integer index
 - Used to access the values
- The Pandas Series has an explicitly defined index associated with the values
 - This explicit index definition gives the Series object additional capabilities
- `data = pd.Series([0.25, 0.5, 0.75, 1.0], index=['a', 'b', 'c', 'd'])`
 - For example, the index does not need to be an integer, but can consist of values of any desired type

- `data['b']`

```
a    0.25
b    0.50
c    0.75
d    1.00
dtype: float64
```



Series as One-Dimensional Array

- A Series provides array-style item selection via the same basic mechanisms as NumPy arrays
 - `data['a':'c']` *# slicing by explicit index*
 - `data[0:2]` *# slicing by implicit integer index*
 - `data[(data > 0.3) & (data < 0.8)]` *# masking*
 - `data[['a', 'e']]` *# fancy indexing*
- Slicing may be the source of the most confusion
 - When you are slicing with an explicit index (i.e., `data['a':'c']`), the final index is included in the slice
 - When you are slicing with an implicit index (i.e., `data[0:2]`), the final index is excluded from the slice

```
a    0.25
b    0.50
c    0.75
d    1.00
dtype: float64
```



Pandas DataFrame

- Pandas DataFrame is a two-dimensional data structure with labeled axes (rows and columns).
- It is created by loading the datasets from existing storage which can be a SQL database, a CSV file or an Excel file.
- It can be created from lists, dictionaries, a list of dictionaries etc.

```
import pandas as pd

df = pd.DataFrame()
print(df)
lst = ['Geeks', 'For', 'Geeks', 'is',
       'portal', 'for', 'Geeks']

df = pd.DataFrame(lst)
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
0
0    Geeks
1      For
2    Geeks
3       is
4  portal
5      for
6    Geeks
```



DataFrame as a Generalized NumPy Array

- We can use a dictionary to construct a single two-dimensional object
 - `population_dict = {'California': 38332521, 'Texas': 26448193}`
 - `population = pd.Series(population_dict)`
 - `area_dict = {'California': 423967, 'Texas': 695662}`
 - `area = pd.Series(area_dict)`
 - `states = pd.DataFrame({'population': population, 'area': area})`
- Like the Series object, the DataFrame has
 - An index attribute that gives access to the index labels
 - `states.index`
 - A columns attribute, which is an Index object holding the column labels
 - `states.columns`
 - A values attribute that provides the raw underlying data array
 - `states.values`

	population	area
California	38332521	423967
Texas	26448193	695662



DataFrame as Two-Dimensional Array

- Pandas again uses the loc, iloc, and ix indexers for DataFrame objects
 - `states.iloc[:3, :2]`
 - `states.loc[:'Texas', :'area']`
 - `states.ix[:3, :'area']`
 - The *ix* indexer allows a hybrid of the other two approaches
- Any of the familiar NumPy-style data access patterns can be used within these indexers
 - For example, we can combine masking and fancy indexing as in the following
 - `states.loc[states.area > 500000, ['population', 'area']]`
- Any of these indexing conventions may also be used to set or modify values
 - `states.iloc[0, 1] = 90`

	population	area
California	38332521	423967
Texas	26448193	695662



DataFrame as a Dictionary

[VanderPlas 2016]

15

- $states['density'] = states['population'] / states['area']$
 - Like with the Series objects, this dictionary-style syntax can also be used to modify the object

	population	area	density
California	38332521	423967	90.413926
Texas	26448193	695662	38.018740



Constructing DataFrame Objects

- `pd.DataFrame(data)`
 - In general, we can create data frame objects with the above line where data can be one of many entities
 - A DataFrame is a collection of Series objects, and a singlecolumn DataFrame can be constructed from a single Series
 - `population_dict = {'California': 38332521, 'Texas': 26448193}`
 - `population = pd.Series(population_dict)`
 - `pd.DataFrame(population, columns=['population'])`
 - Any list of dictionaries can be made into a DataFrame
 - `data = [{'a': 1, 'b': 2}, {'a': 3, 'b': 4}]`
 - `pd.DataFrame(data)`
 - Even if some keys in the dictionary are missing, Pandas will fill them in with NaN (i.e., “not a number”) values
 - `pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])`
 - As we saw before, a DataFrame can be constructed from a dictionary of Series objects as well
 - `pd.DataFrame({'population': population, 'area': area})`
 - Given a two-dimensional array of data, we can create a DataFrame with any specified column and index names
 - `pd.DataFrame(np.random.rand(3, 2), columns=['foo', 'bar'], index=['a', 'b', 'c'])`



UFuncs: Index Preservation

- Any NumPy ufunc will work on Pandas Series and DataFrame objects
 - Because Pandas is designed to work with NumPy
- If we apply a NumPy ufunc on either of these objects, the result will be another Pandas object with the indices preserved
 - `ser = pd.Series([1, 2, 3])`
 - `np.exp(ser)`
 - `df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])`
 - `np.sin(df * np.pi / 4)`

		A	B
0	2.718282		
1	7.389056		
2	20.085537		
dtype:	float64		

0	0.707107	1.000000e+00
1	0.707107	1.224647e-16



UFuncs: Index Alignment

- Pandas aligns indices for binary operations on two Series/DataFrame objects
 - This is very convenient when you are working with incomplete data
 - `area = pd.Series({'Alaska': 1723337, 'Texas': 695662, 'California': 423967})`
 - `population = pd.Series({'California': 38332521, 'Texas': 26448193, 'New York': 19651127})`
 - `population / area`
 - The resulting array contains the union of indices of the two input arrays
 - `area.index | population.index`
- Any item for which one or the other does not have an entry is marked with *NaN*
 - Or “Not a Number”

```
Alaska          NaN
California      90.413926
New York        NaN
Texas           38.018740
dtype: float64
```



UFuncs: Index Alignment

- If using *NaN* values is not the desired behavior, we can modify the fill value
 - Using appropriate object methods in place of the operators
- For example, calling *A.add(B)* is equivalent to calling *A + B*
 - But it allows optional explicit specification of the fill value for any elements in *A* or *B* that might be missing
 - *A = pd.Series([2, 4, 6], index=[0, 1, 2])*
 - *B = pd.Series([1, 3, 5], index=[1, 2, 3])*
 - *A.add(B, fill_value=0)*

```
0      2.0
1      5.0
2      9.0
3      5.0
dtype: float64
```

Python operator	Pandas method(s)
+	add()
-	sub(), subtract()
*	mul(), multiply()
/	truediv(), div(), divide()
//	floordiv()
%	mod()
**	pow()



Handling Missing Data with Pandas

- Real-world data is rarely clean and homogeneous
 - In particular, many interesting datasets will have some amount of data missing
 - Different data sources may indicate missing data in different ways
- Pandas chose to use sentinels for missing data using two already-existing Python null values
 - The Python None object
 - The special floatingpoint NaN value of NumPy
- Pandas treats *None* and *NaN* as essentially interchangeable
 - For indicating missing or null values



Operating on Null Values

- There are several useful methods for detecting, removing, and replacing null values in Pandas data structures
 - *isnull()*
 - Generate a Boolean mask indicating missing values
 - *notnull()*
 - Opposite of *isnull()*
 - *dropna()*
 - Return a filtered version of the data
 - *fillna()*
 - Return a copy of the data with missing values filled or imputed



Detecting Null Values

- Pandas data structures have two useful methods for detecting null data
 - `isnull()`
 - `notnull()`
- Either one will return a Boolean mask over the data
 - `data = pd.Series([1, np.nan, 'hello', None])`
 - `data.isnull()`
 - `data[data.notnull()]`

```
0      1
1    None
2  hello
3    None
dtype: object
```

```
0    False
1     True
2    False
3     True
dtype: bool
```

```
0      1
2  hello
dtype: object
```



Dropping Null Values

- `data.dropna()`
 - For a Series, the result is straightforward
 - For a DataFrame, we can only drop full rows or full columns
 - By default, `dropna()` will drop all rows in which any null value is present
 - `df = pd.DataFrame([[1, np.nan, 2], [2, 3, 5], [np.nan, 4, 6]])`
 - `df.dropna()`
 - Alternatively, you can drop NA values along a different axis
 - `df.dropna(axis='columns')`
- You might rather be interested in dropping rows or columns with all NA values
 - You can also specify `how='all'`, which will only drop rows/columns that are all null values
 - `df[3] = np.nan`
 - `df.dropna(axis='columns', how='all')`
 - The `thresh` parameter lets you specify a minimum number of non-null values for the row/column to be kept
 - `df.dropna(axis='rows', thresh=3)`



Filling Null Values

[VanderPlas 2016]

<https://towardsdatascience.com/mean-median-and-mode-what-are-they-and-when-should-you-use-them-ed3949aa142>

24

- Sometimes rather than dropping NA values, we would rather replace them with a valid value
 - This value might be a single number like zero
 - It might be some sort of imputation or interpolation from the good values
- We can fill NA entries with a single value, such as zero
 - `data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'))`
 - `data.fillna(0)`
 - We can specify a forward-fill to propagate the previous value forward
 - `data.fillna(method='ffill')`
 - For DataFrames, the options are similar
 - But we can also specify an axis along which the fills take place
 - `df.fillna(method='ffill', axis=1)`

```
a    1.0
b    NaN
c    2.0
d    NaN
e    3.0
dtype: float64
```

```
a    1.0
b    0.0
c    2.0
d    0.0
e    3.0
dtype: float64
```



Combining Datasets: Concat and Append

- The `pd.concat()` function can be used for a simple concatenation of Series or DataFrame objects
 - `ser1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])`
 - `ser2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])`
 - `pd.concat([ser1, ser2])`
 - `df1 = pd.DataFrame([[1, 2], [3, 4]])`
 - `df2 = pd.DataFrame([[5, 6], [7, 8]])`
 - `pd.concat([df1, df2])`
 - By default, the concatenation takes place row-wise within the DataFrame (i.e., `axis=0`)
 - `pd.concat([df1, df2], axis=1)`
- One important difference between `np.concatenate` and `pd.concat` is that Pandas concatenation preserves indices
 - Even if the result will have duplicate indices

```
1    A
2    B
3    C
4    D
5    E
6    F
dtype: object
```

	0	1
0	1	2
1	3	4
0	5	6
1	7	8



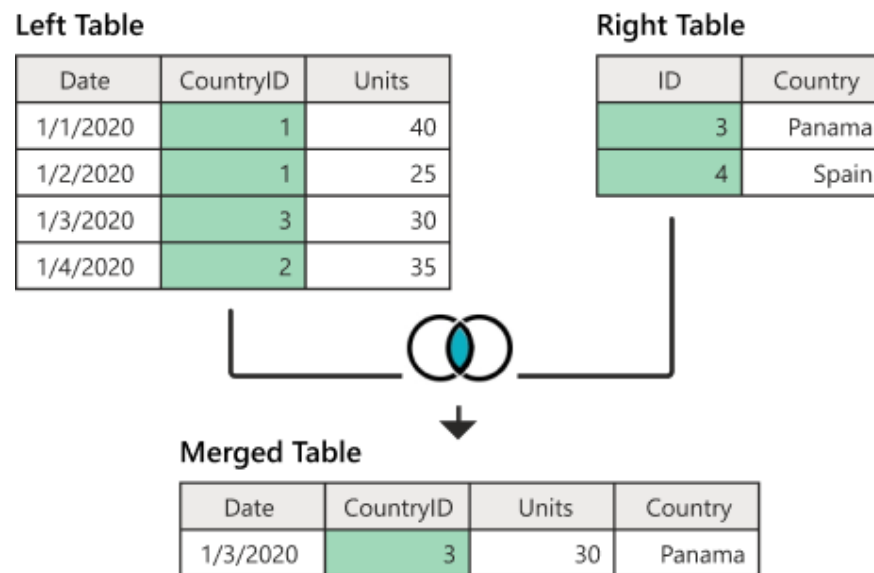
Combining Datasets: Merge and Join

[VanderPlas 2016]

<https://docs.microsoft.com/en-us/power-query/merge-queries-inner>

26

- One essential feature offered by Pandas is its high-performance, in-memory join and merge operations
- The `pd.merge()` function implements a number of types of joins
 - One-to-one
 - Many-to-one
 - Many-to-many





Specification of The Merge Key

- `pd.merge()` looks for one or more matching column names between the two inputs
 - We can explicitly specify the name of the key column using the `on` keyword
 - Which takes a column name or a list of column names
 - `df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue'], 'group': ['Accounting', 'Engineering', 'Engineering', 'HR']})`
 - `df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue'], 'hire_date': [2004, 2008, 2012, 2014]})`
 - `pd.merge(df1, df2, on='employee')`

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014



Specification of The Merge Key

- We can use the `left_on` and `right_on` keywords to specify the two column names to merge on
 - `df3 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'], 'salary': [70000, 80000, 120000, 90000]})`
 - `pd.merge(df1, df3, left_on="employee", right_on="name")`
- The result has a redundant column that we can drop if desired
 - `pd.merge(df1, df3, left_on="employee", right_on="name").drop('name', axis=1)`

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

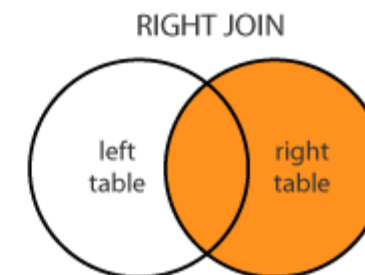
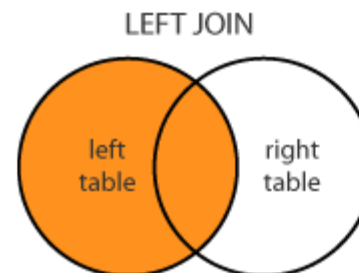
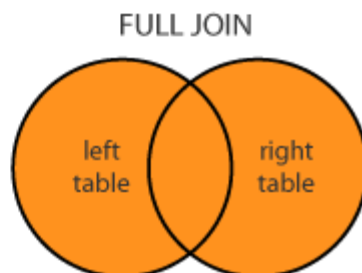
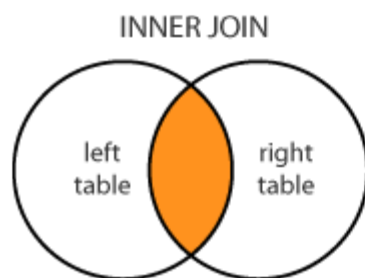
	name	salary
0	Bob	70000
1	Jake	80000
2	Lisa	120000
3	Sue	90000

	employee	group	name	salary
0	Bob	Accounting	Bob	70000
1	Jake	Engineering	Jake	80000
2	Lisa	Engineering	Lisa	120000
3	Sue	HR	Sue	90000



Specifying Set Arithmetic for Joins

- By default, the result contains the intersection of the two sets of inputs
 - Which is known as an inner join
 - `df6 = pd.DataFrame({'name': ['Peter', 'Paul', 'Mary'], 'food': ['fish', 'beans', 'bread']}, columns=['name', 'food'])`
 - `df7 = pd.DataFrame({'name': ['Mary', 'Joseph'], 'drink': ['wine', 'beer']}, columns=['name', 'drink'])`
 - `pd.merge(df6, df7, how='inner')`
- Other options for the `how` keyword are `'outer'`, `'left'`, and `'right'`
 - An outer join returns a join over the union of the input columns, and fills in all missing values with NAs





Simple Aggregation in Pandas

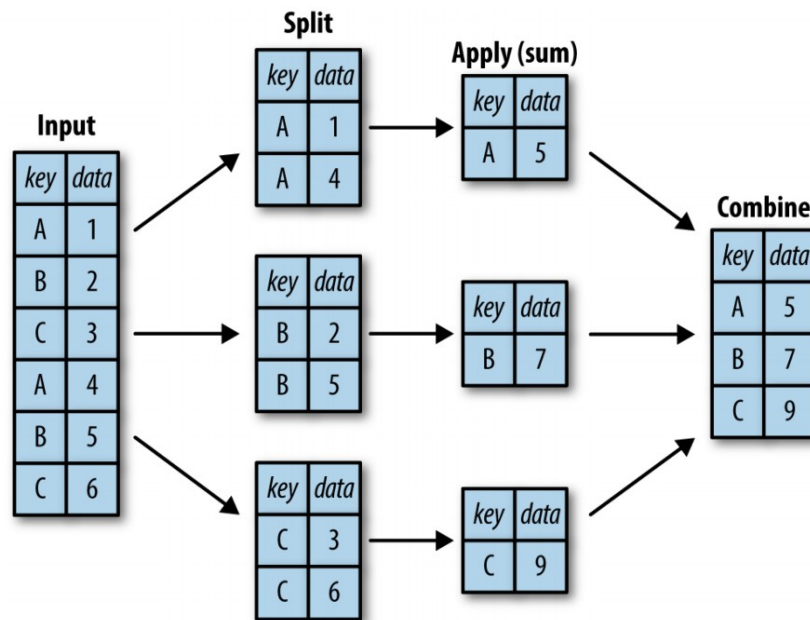
- For a Pandas Series the aggregates return a single value
 - `ser = pd.Series([1, 2, 3])`
 - `ser.sum()`
- For a DataFrame, by default the aggregates return results within each column
 - `df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})`
 - `df.mean()`
 - By specifying the axis argument, you can instead aggregate within each row
 - `df.mean(axis='columns')`
- `describe()` computes several common aggregates for each column
 - `df.describe()`

Aggregation	Description
<code>count()</code>	Total number of items
<code>first(), last()</code>	First and last item
<code>mean(), median()</code>	Mean and median
<code>min(), max()</code>	Minimum and maximum
<code>std(), var()</code>	Standard deviation and variance
<code>mad()</code>	Mean absolute deviation
<code>prod()</code>	Product of all items
<code>sum()</code>	Sum of all items



GroupBy: Split, Apply, Combine

- Often, we would prefer to aggregate conditionally on some label or index
 - This is implemented in the so-called groupby operation
- The name “*group by*” comes from a command in the SQL database language
 - Think of it as split, apply, combine



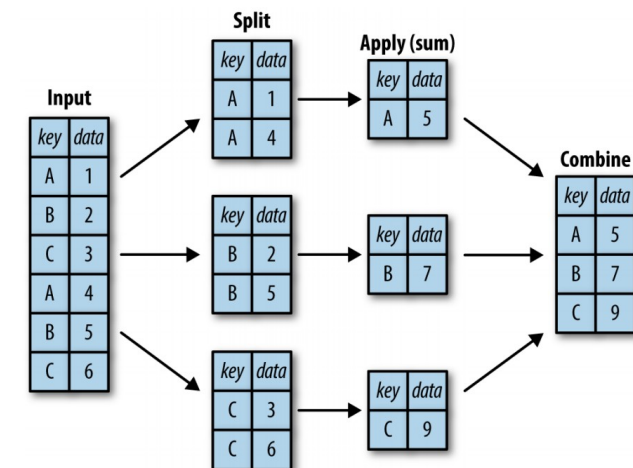


GroupBy: Split, Apply, Combine

[VanderPlas 2016]

32

- Groupby steps
 - The split step involves breaking up and grouping a DataFrame depending on the value of the specified key
 - The apply step involves computing some function, usually an aggregate, transformation, or filtering, within the individual groups
 - The combine step merges the results of these operations into an output array
- The result of groupby is not a set of DataFrames, but a DataFrameGroupBy object
 - `df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'], 'data': range(6)}, columns=['key', 'data'])`
 - `df.groupby('key')`
 - To produce a result, we can apply an aggregate to this DataFrameGroupBy object
 - `df.groupby('key').sum()`





Column Indexing

- The GroupBy object supports column indexing in the same way as the DataFrame
 - And returns a modified GroupBy object
 - `df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'], 'data1': range(6), 'data2': range(7, 13)}, columns=['key', 'data1', 'data2'])`
 - `df.groupby('key')['data2'].mean()`

	key	data1	data2
0	A	0	7
1	B	1	8
2	C	2	9
3	A	3	10
4	B	4	11
5	C	5	12

```
key
A    8.5
B    9.5
C   10.5
Name: data2, dtype: float64
```



Some Other Series/DataFrame Methods

https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html

<https://towardsdatascience.com/powerful-cool-tricks-in-pandas-every-data-analyst-should-know-73>

34

- `df.head()`
- `df.tail()`
- `df.to_numpy()`
- `df.sort_values(by="B")`
- `df.apply(lambda x: x + 10)`
- `s.value_counts()`
- `s.str.lower()`



Pandas Plotting Functions

- We can group the plotting functions available in Pandas into two groups
 - *DataFrame.plot*
 - *pandas.plotting*



Dataframe.plot

- DataFrame.plot is used extensively for quick exploratory data analysis
 - `DataFrame.boxplot([column, by, ax, ...])`
 - `DataFrame.hist([column, by, grid, ...])`
 - `DataFrame.plot([x, y, kind, ax, ....])`
 - `DataFrame.plot.area([x, y])`
 - `DataFrame.plot.bar([x, y])`
 - `DataFrame.plot.barh([x, y])`
 - `DataFrame.plot.box([by])`
 - `DataFrame.plot.density([bw_method, ind])`
 - `DataFrame.plot.hexbin(x, y[, C, ...])`
 - `DataFrame.plot.hist([by, bins])`
 - `DataFrame.plot.kde([bw_method, ind])`
 - `DataFrame.plot.line([x, y])`
 - `DataFrame.plot.pie(**kwargs)`
 - `DataFrame.plot.scatter(x, y[, s, c])`



Pandas.plotting

- Pandas.plotting functions are used for specific statistical and quantitative analysis
 - `pandas.plotting.andrews_curves`
 - `pandas.plotting.autocorrelation_plot`
 - `pandas.plotting.bootstrap_plot`
 - `pandas.plotting.boxplot`
 - `pandas.plotting.lag_plot`
 - `pandas.plotting.parallel_coordinates`
 - `pandas.plotting.plot_params`
 - `pandas.plotting.radviz`
 - `pandas.plotting.scatter_matrix`
 - `pandas.plotting.table`



Example: Line Chart

[Rahman 2021]

38

- Consider the following toy dataset

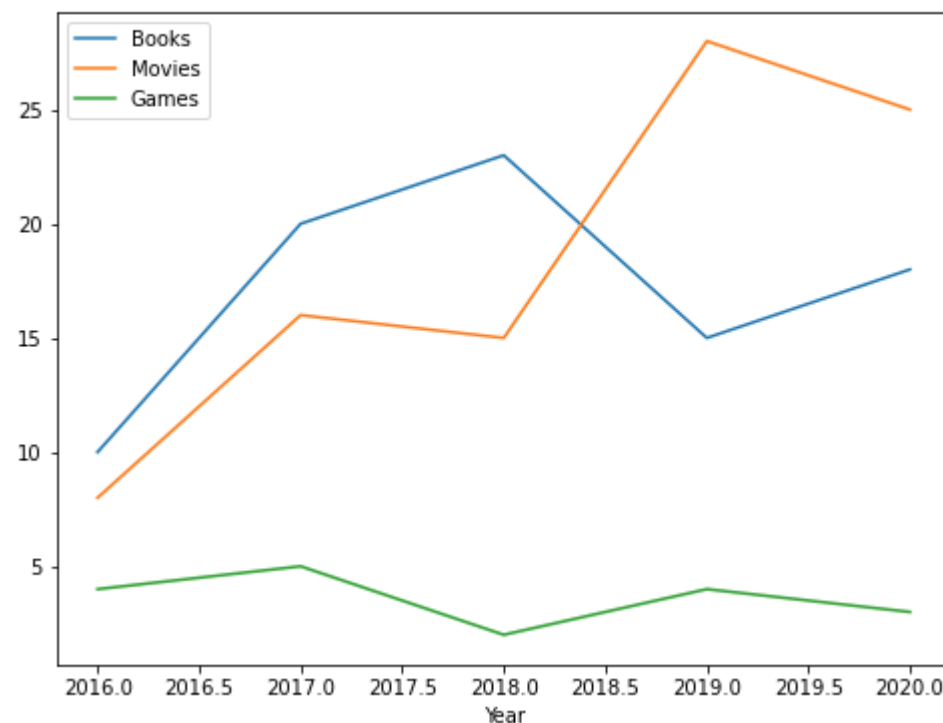
```
import pandas as pd

df = pd.DataFrame({"Year": [2016, 2017, 2018, 2019, 2020],
                  "Books": [10, 20, 23, 15, 18],
                  "Movies": [8, 16, 15, 28, 25],
                  "Games": [4, 5, 2, 4, 3]})

df.head()
```

	Year	Books	Movies	Games
0	2016	10	8	4
1	2017	20	16	5
2	2018	23	15	2
3	2019	15	28	4
4	2020	18	25	3

```
df.plot.line(y=["Books", "Movies", "Games"], x="Year", figsize=(8, 6));
```





Example: Subplots

[Rahman 2021]

39

- Consider the following toy dataset

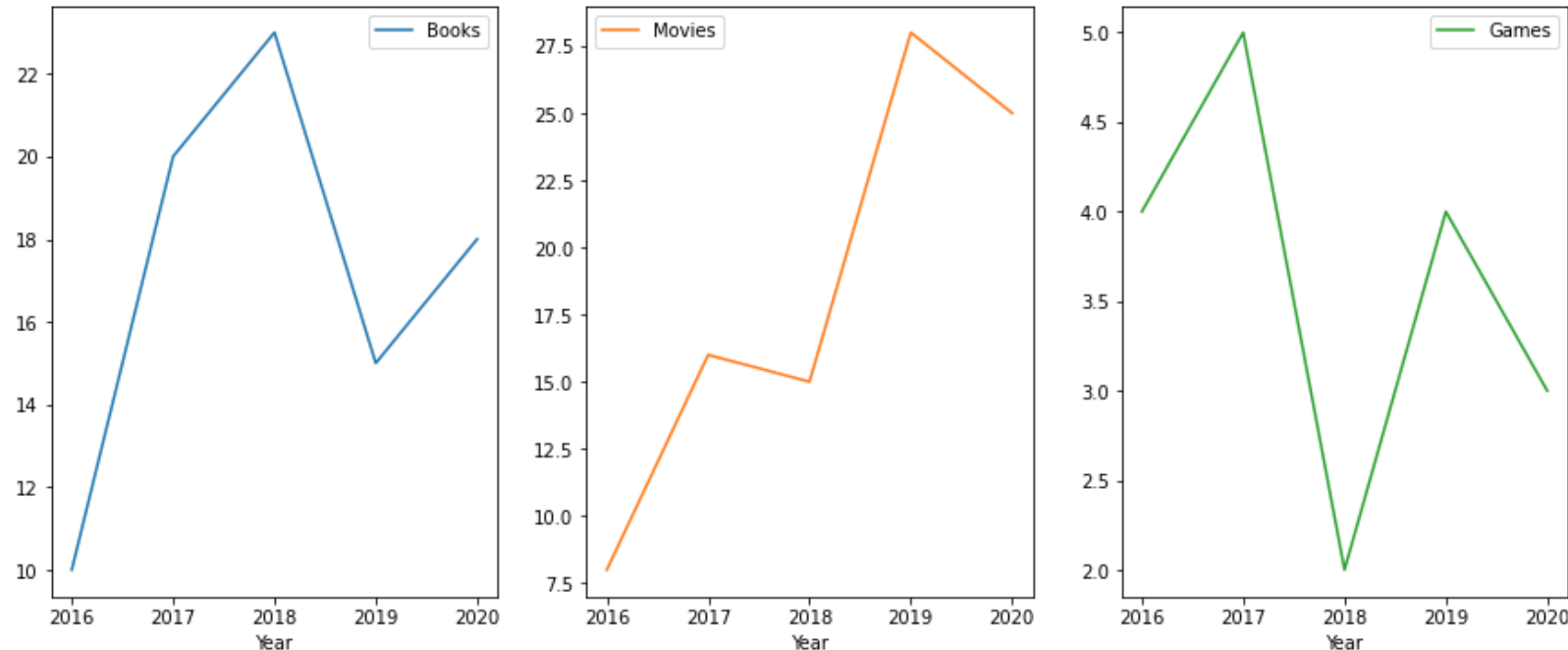
```
import pandas as pd

df = pd.DataFrame({"Year": [2016, 2017, 2018, 2019, 2020],
                  "Books": [10, 20, 23, 15, 18],
                  "Movies": [8, 16, 15, 28, 25],
                  "Games": [4, 5, 2, 4, 3]})

df.head()
```

	Year	Books	Movies	Games
0	2016	10	8	4
1	2017	20	16	5
2	2018	23	15	2
3	2019	15	28	4
4	2020	18	25	3

```
df.plot.line(y=["Books", "Movies", "Games"], x="Year", figsize=(15, 6),
            subplots=True, layout=(1, 3));
```





Example: Bar Chart

[Rahman 2021]

40

- Consider the following toy dataset

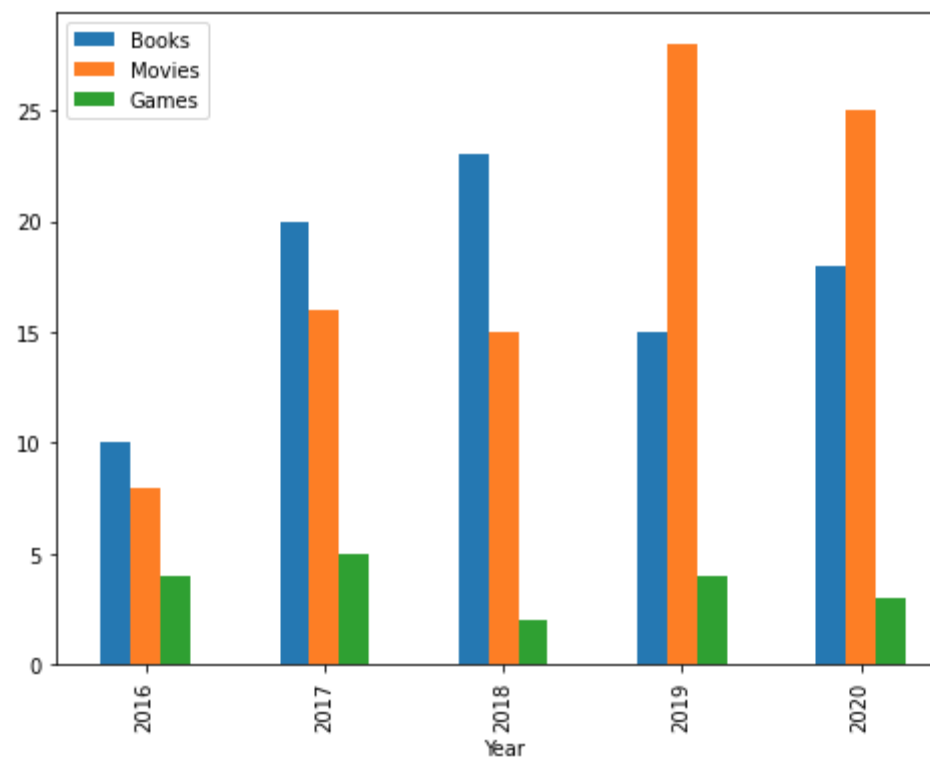
```
import pandas as pd

df = pd.DataFrame({"Year": [2016, 2017, 2018, 2019, 2020],
                  "Books": [10, 20, 23, 15, 18],
                  "Movies": [8, 16, 15, 28, 25],
                  "Games": [4, 5, 2, 4, 3]})

df.head()
```

	Year	Books	Movies	Games
0	2016	10	8	4
1	2017	20	16	5
2	2018	23	15	2
3	2019	15	28	4
4	2020	18	25	3

```
df.plot.bar(y=["Books", "Movies", "Games"], x="Year", figsize=(8, 6));
```





Example: Scatter Plot

[Rahman 2021]

41

- Consider the following toy dataset

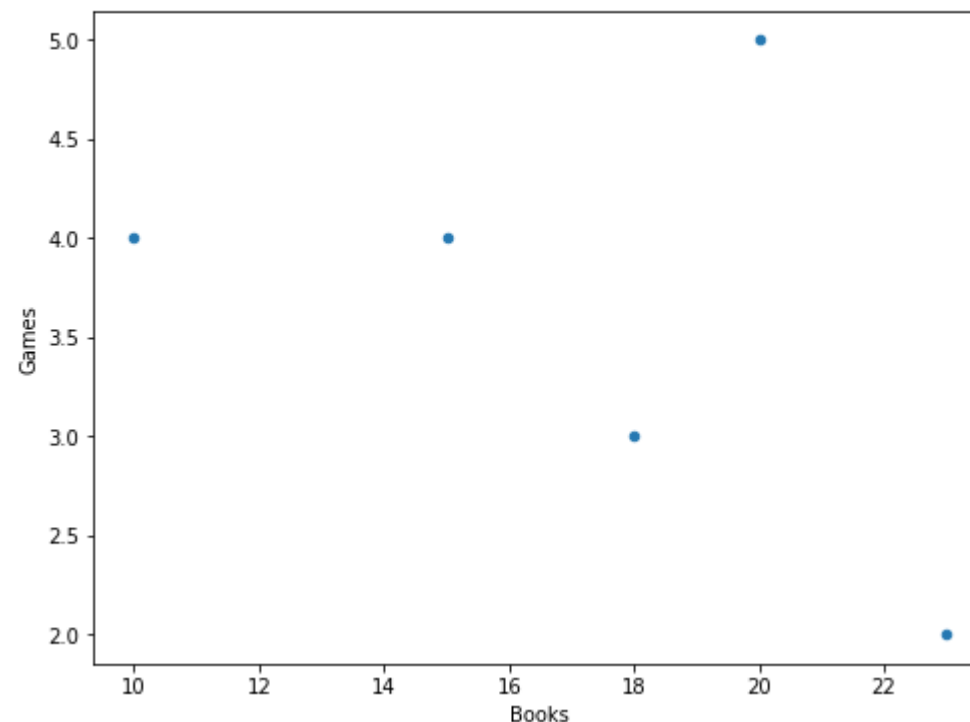
```
import pandas as pd

df = pd.DataFrame({"Year": [2016, 2017, 2018, 2019, 2020],
                  "Books": [10, 20, 23, 15, 18],
                  "Movies": [8, 16, 15, 28, 25],
                  "Games": [4, 5, 2, 4, 3]})

df.head()
```

	Year	Books	Movies	Games
0	2016	10	8	4
1	2017	20	16	5
2	2018	23	15	2
3	2019	15	28	4
4	2020	18	25	3

```
df.plot.scatter(x="Books", y="Games", figsize=(8, 6));
```





Changing the Plotting Backend

- The default plotting backend is Matplotlib
 - `pd.options.plotting.backend = "matplotlib"`
- But it can be changed in different ways
 - `pd.options.plotting.backend = "plotly"`
 - `pd.set_option("plotting.backend", "hvplot")`



Pandas Cheat Sheet

https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

<https://towardsdatascience.com/essential-techniques-to-style-pandas-dataframes-8796c74c9ff3>

Essential Techniques to Style pandas DataFrames

Data Wrangling

with pandas Cheat Sheet
<http://pandas.pydata.org>

Pandas API Reference Pandas User Guide

Creating DataFrames

```
df = pd.DataFrame({
    "a": [4, 5, 6],
    "b": [7, 8, 9],
    "c": [10, 11, 12]},
    index=[1, 2, 3])
```

```
df = pd.DataFrame(
    [[4, 7, 10],
     [5, 8, 11],
     [6, 9, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

```
df = pd.DataFrame(
    [[4, 5, 6],
     [7, 8, 9],
     [10, 11, 12]],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

```
df = pd.DataFrame(
    [{"a": 4, "b": 7, "c": 10},
     {"a": 5, "b": 8, "c": 11},
     {"a": 6, "b": 9, "c": 12}],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

```
df = pd.DataFrame(
    [{"a": 4, "b": 7, "c": 10},
     {"a": 5, "b": 8, "c": 11},
     {"a": 6, "b": 9, "c": 12}],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

```
df = pd.DataFrame(
    [{"a": 4, "b": 7, "c": 10},
     {"a": 5, "b": 8, "c": 11},
     {"a": 6, "b": 9, "c": 12}],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

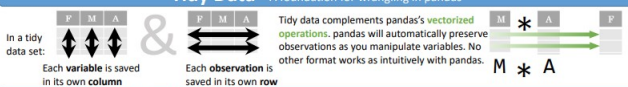
```
df = pd.DataFrame(
    [{"a": 4, "b": 7, "c": 10},
     {"a": 5, "b": 8, "c": 11},
     {"a": 6, "b": 9, "c": 12}],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

```
df = pd.DataFrame(
    [{"a": 4, "b": 7, "c": 10},
     {"a": 5, "b": 8, "c": 11},
     {"a": 6, "b": 9, "c": 12}],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

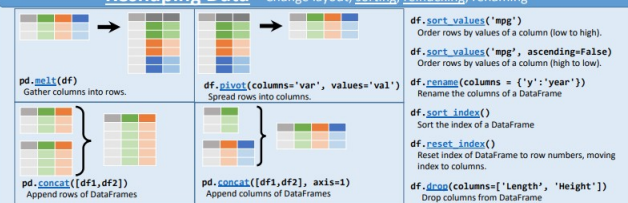
```
df = pd.DataFrame(
    [{"a": 4, "b": 7, "c": 10},
     {"a": 5, "b": 8, "c": 11},
     {"a": 6, "b": 9, "c": 12}],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

```
df = pd.DataFrame(
    [{"a": 4, "b": 7, "c": 10},
     {"a": 5, "b": 8, "c": 11},
     {"a": 6, "b": 9, "c": 12}],
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Tidy Data – A foundation for wrangling in pandas



Reshaping Data – Change layout, sorting, reindexing, renaming



Subset Observations - rows

```
df[df.Length > 7]
df.drop_duplicates()
df.sample(frac=0.5)
df.drop_duplicates(subset='a')
df.sort_values('a')
```

Subset Variables - columns

```
df[['width', 'length', 'species']]
df.drop('width', axis=1)
df.filter(regex='^sp')
df.filter(regex='^sp', axis=1)
```

Subsets - rows and columns

```
df.loc[1:2]
df.iloc[1:2]
df.ix[1:2]
df.iat[1, 0]
```

Using query

```
df.query('Length > 7')
df.query('Length > 7 and Width < 8')
df.query('Name.str.startswith("abc")')
```

Logic in Python (and pandas)

Logic in Python (and pandas)	regex (Regular Expressions) Examples
Less than	<
Greater than	>
Equals	==
Less than or equal	<=
Greater than or equal	>=

Summarize Data

```
df['a'].value_counts()
df.agg([sum, min, max])
df.describe()
```

Handling Missing Data

```
df.isna()
df.dropna()
df.fillna(value)
```

Make New Columns

```
df['Volume'] = df.Length * df.Height * df.Depth
df.assign(new_col=1)
```

Group Data

```
df.groupby('by="col"')
df.groupby(level="ind")
df.agg([sum, min, max])
```

Windows

```
df.rolling(window=3)
df.ewm(span=3)
```

Plotting

```
df.plot()
df.plot.scatter(x='a', y='b')
```

Combine Data Sets

```
df1.append(df2)
df1.join(df2)
```

Display Options

```
pd.set_option('display.max_columns', None)
pd.set_option('display.max_colwidth', None)
```

General Parameters

```
axis = 0
axis = 1
subset
```

Formatting

```
df.style.set_caption(...)
df.style.hide_index()
```

Styling Properties

```
df.style.set_properties(**{"background-color": ...,
                             "color": ...,
                             "border": "0.5px solid ..."})
```

Built-in Styling

```
df.style.highlight_null(null_color = ...)
df.style.highlight_min(color = ...)
df.style.highlight_max(color = ...)
```

Custom Styling

```
def custom_styling(val):
    # Your custom styling code here, e.g.
    color = "red" if val < 0 else "black"
    return f"color: {color}"
```

Export to Excel

```
pip install openpyxl
df.style.to_excel([file_name], engine = "openpyxl")
```